



Status and Plans of HTCondor Usage in CMS

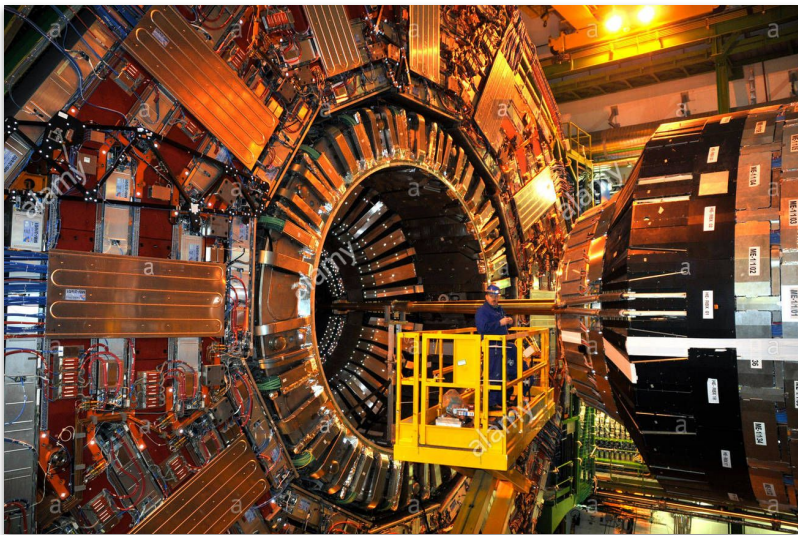
Marco Mascheroni

On behalf of the CMS Submission Infrastructure team at CERN

Presentation layout

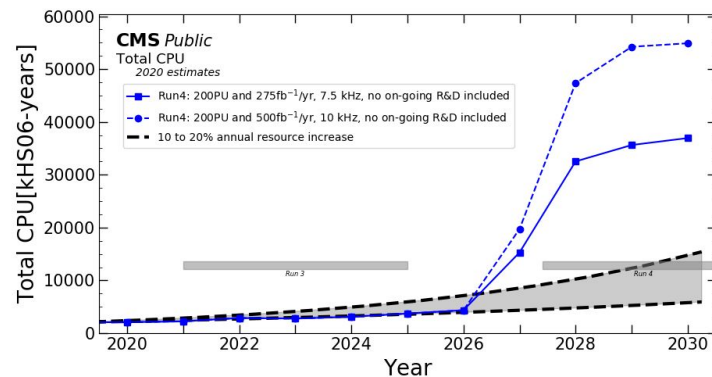
- Introduction to the CMS experiment at CERN
- Description of the HTCondor pools
 - How and what we use
 - Monitoring
- Challenges
 - Efficiency
 - Scalability
- Future Directions

The CMS experiment at CERN



- General-purpose experiment running at the LHC at CERN
- Acts like a giant camera that takes 3D pictures of proton collisions (1 MB per pic at ~1 KHz during data taking years)

- Experimental data is stored, distributed, reconstructed, and analyzed, comparing to simulated data (Monte Carlo)
- Current estimates predict ~4x gap in CPU needs to address resource evolution by the High-Luminosity LHC era (starting in 2027)

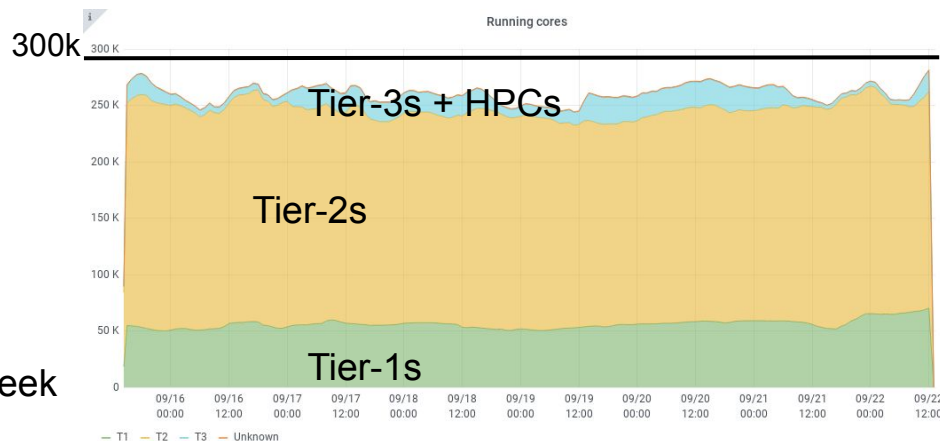


[CMS computing public results](#)

The CMS Submission Infrastructure Group



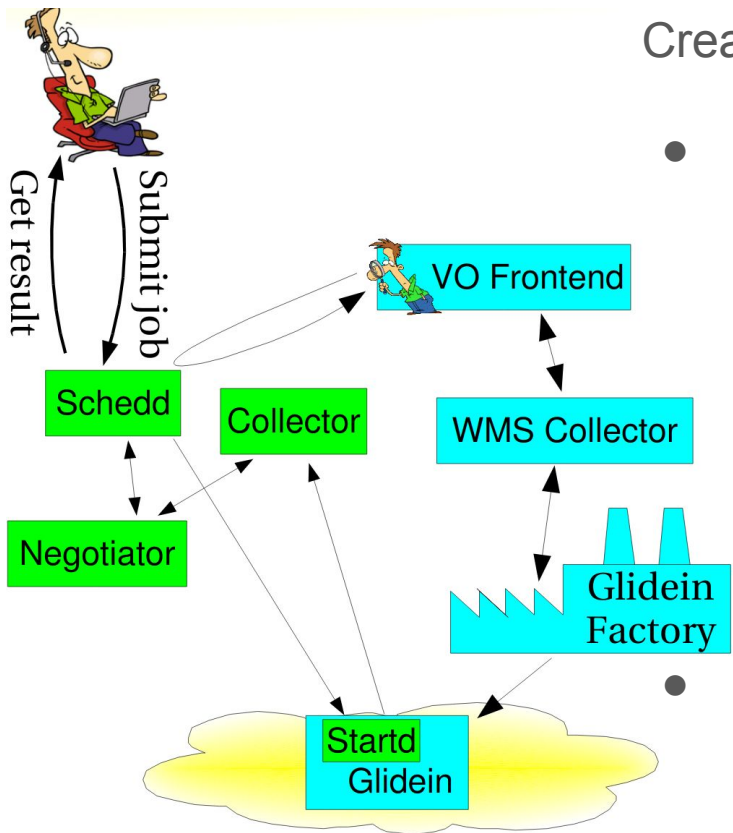
- Part of CMS Offline and computing in **charge** of:
 - **Running the distributed infrastructure** where all reconstruction, simulation, and analysis of physics data takes place
 - Communicate CMS **priorities to the development teams** of glideinWMS and HTCondor
- In practice:
 - We operate a (vanilla) HTCondor pool peaking at **250k/300k cores distributed over 70 Grid sites, plus non-Grid resources (HLT farm, HPC sites, Cloud)**
 - We run the software needed to create it on the flight (**glideinWMS**)
 - We regularly hold **meetings with HTCondor and glideinWMS developers** where we discuss
 - Current operational limitations
 - Feature requests
 - Future scale requirements



The global pool cores last week

Dynamic HTCondor pool(s)

Creation of the HTCondor pools:



- Through glideinWMS and **pilot jobs**

- To access **regular Grid resources** (the majority of the CMS resources)
- The glideinWMS **frontend looks** at the user job **pressure**
- **Matchmaking** with resources defined in the factory (CEs)
- **Frontend makes** pilot job submission **requests** to the factory
 - using the WMS HTCondor collector
- **Factory** then **sends pilot** jobs using condor Grid universe
- **Pilot job** will then **start the HTCondor startd** which will connect to the pool

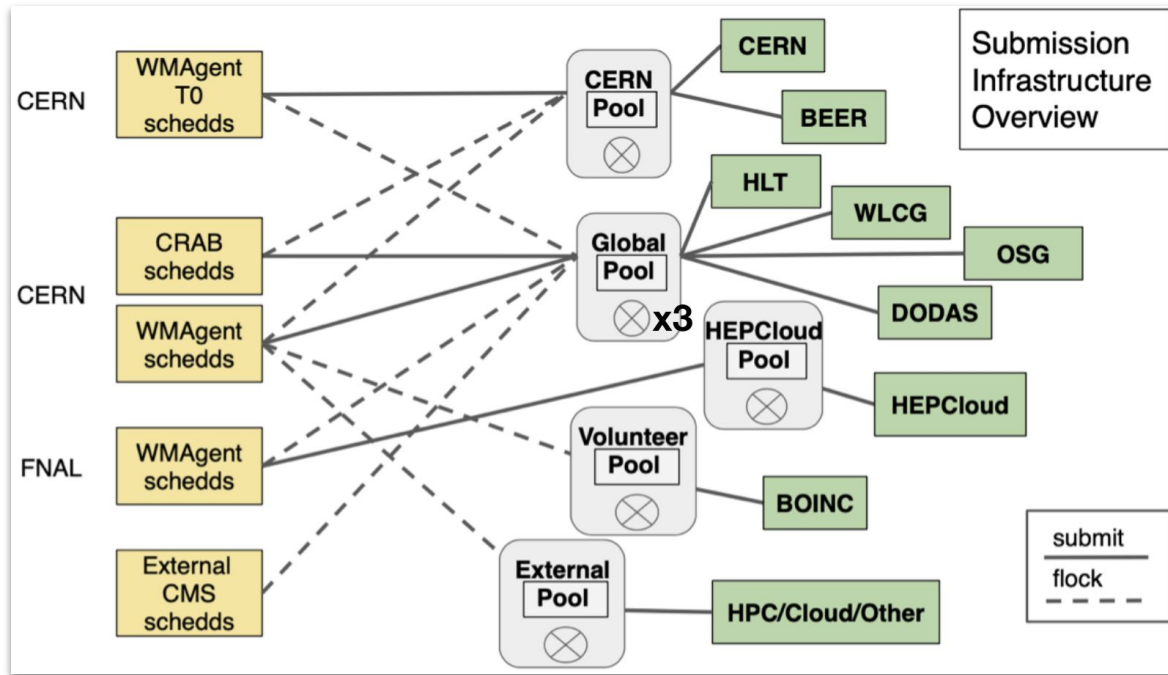
- The *vacuum* model

- Pilot jobs manually launched when resource are available
- Suitable for particular types of resources where using a CE is not possible (e.g.: HLT farm, DODAS project, CMS@HOME)

The CMS HTCondor pools landscape



- Two sets of workflow managers
 - CRAB: user jobs from more than 400 physicists
 - WMAgent: central production needs
- Model evolved to use **federated pools**, with extensive use of **flocking**
- **Global pool** is the biggest and most important one
 - 250k/300k cores
 - 52 schedds
 - 3 negotiators



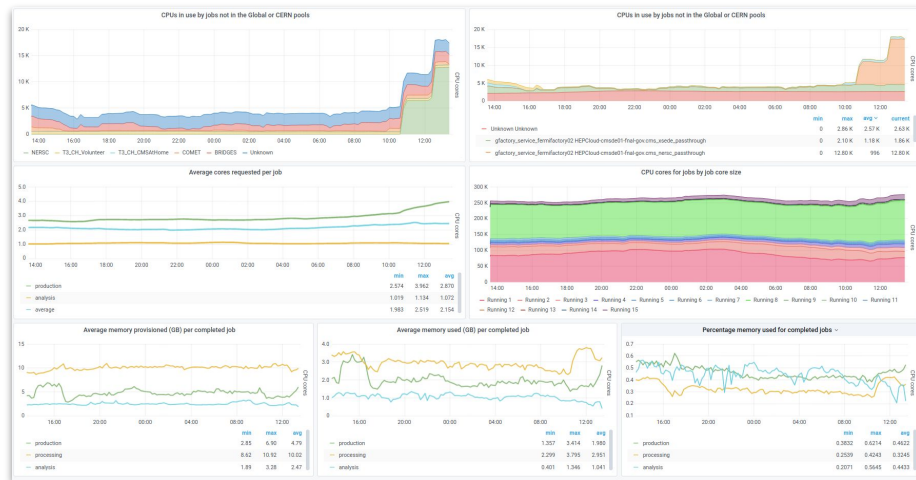
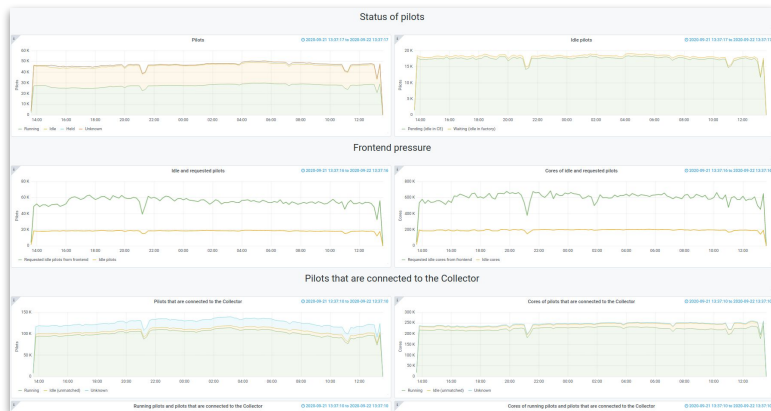
One of the biggest and most complex collection of HTCondor pools in the world

HTCondor features in use by CMS

- **Flocking** used to migrate jobs from the pool where they naturally belong to other pools where they can run
- **Dagman** used by CRAB to manage jobs **resubmission and transfer** of output files
- **JobRouter** used to adjust on the fly user job requirements
 - *Overflow*: jobs idle for a long time on a site
 - Tuning of jobs requirements (i.e.: walltime)
- **Custom start expressions**
 - Site admins can specify in their local resource configuration extra requirements for their jobs; useful for expansion of the pool to **non-standard resources**
 - E.g.: just run GEN-SIM jobs on opportunistic resources
- **Resizable jobs** have been introduced to improve **resource usage efficiency**
 - Jobs will match a range of resources (CPUs/Memory)
 - A job can reconfigure itself at runtime based on the resources allocated

Monitoring tools

- Migrated all monitoring tools to the MonIt infrastructure
 - Mostly relying on ElasticSearch + Grafana instances provided by CERN IT
- Submission infrastructure: six different dashboards with over 100 plots
 - **Slots** usage, GWMS **factory** statistics, **payload** information, **schedulers** management, **negotiator** and **collector** statistics
- Alarms set to help us operating the infrastructure at best



Challenges

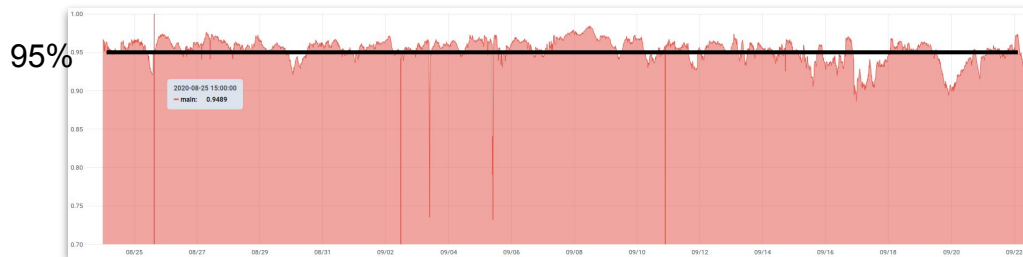
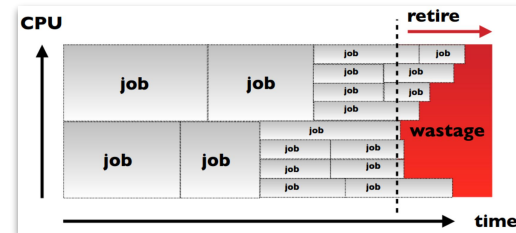
Scheduling needs

Need to combine different needs:

- Users want their workflows to complete in a predictable amount of time with limited manual intervention
 - Either centrally launched with WMAgent, or personal CRAB workflows
 - Don't usually care about efficiency or cost, just time to completion and success rate
 - Sometimes there are *crazy* requirements (once CPU 20Gb of RAM)
- Resources used all the time efficiently
 - Need to be CPU efficient
- Priorities must be respected (*higher prio work should complete sooner/faster than lower prio*)

Scheduling Efficiency

- The Global pool is a multi-core pool with dynamic provisioning of slots (CPUs and Memory)
- Partitionable slots can become fragmented over time
- Focus on making sure pilots are efficiently scheduled
 - Optimizing retiring time
 - Memory starvation (p-slots using all the memory before CPUs are allocated)
 - Making sure there is always work
- Effort trying to reduce any remaining sources of inefficiency

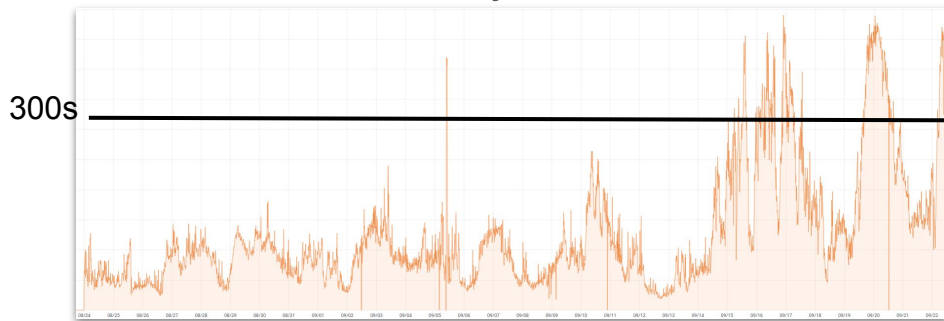


Slot usage in the last 30 days

Negotiation time

- Negotiation time is one of the critical aspects of the pool performance
- Possible source of inefficiency
 - On the WN, startds processes only wait for 10 minutes for a user job
 - Results in CPU wastages
- Target: below 300 seconds

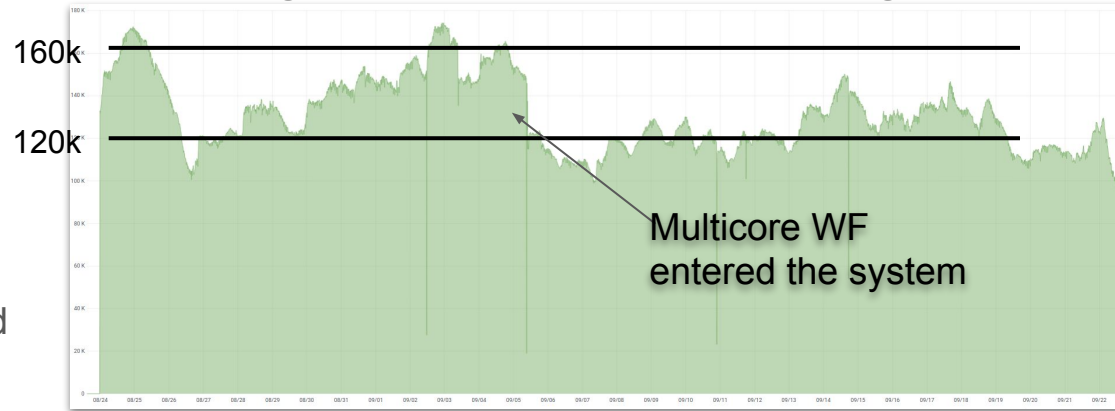
(Lack of) Scalability is a possible source of inefficiency



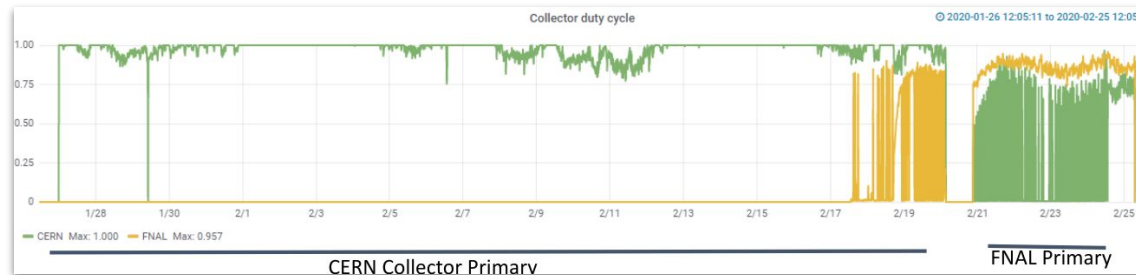
T2 Negotiation time in the last 30 days

Scalability: the Central Manager bottleneck

- Number of dynamic slots is the main scaling driver for Collector and Negotiator
 - Results in UDP buffer saturation
 - Oscillations according to the CMS workload mix composition
- We regularly run scale tests trying to push the pool limits
 - Recently redirected queries used by monitoring tools on to a separate collector



Number of dynamic slots the last month



Conclusions and future works

- HL-LHC will push us in trying to make use of all available compute power
 - Federated pools to access wide variety of resources (HPC through HEPCloud, HLT, etc.)
 - Pushing boundaries of single pools
- Efficiency and scalability are key aspects
 - Would like to explore more task tuning (through JobRouter) to modify job resource requests and improve the scheduling efficiency
- Cloud and HPC integration
- Access and exploitation of heterogeneous resources (e.g. CPU+GPU)

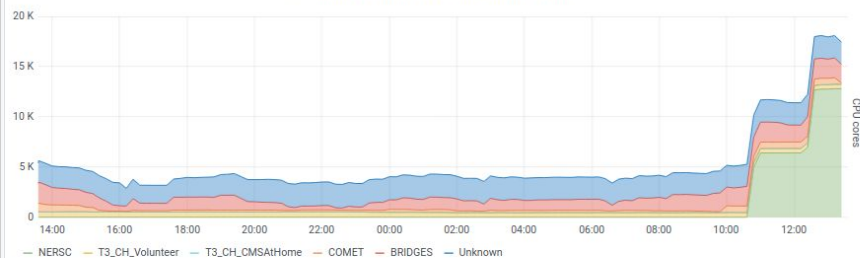
Backup slides

Monitoring the CMS Global Pool (I)

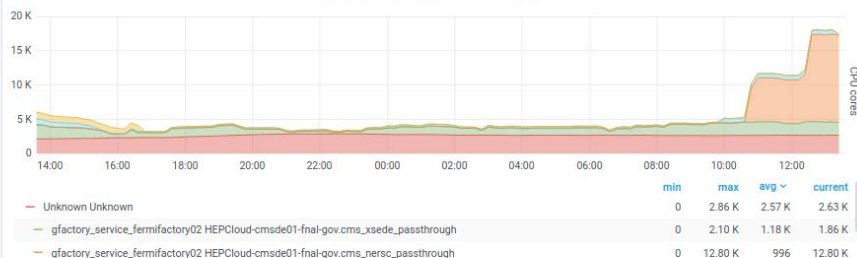


Monitoring the CMS Global Pool (II)

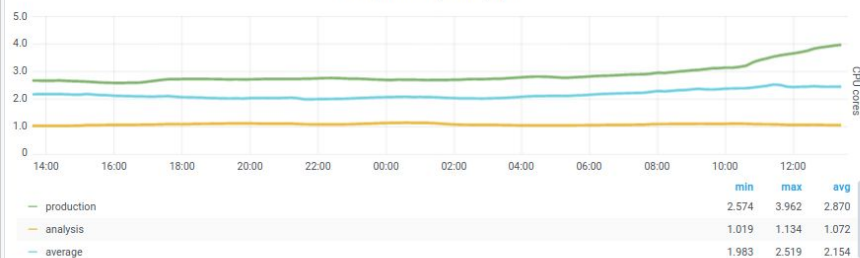
CPUs in use by jobs not in the Global or CERN pools



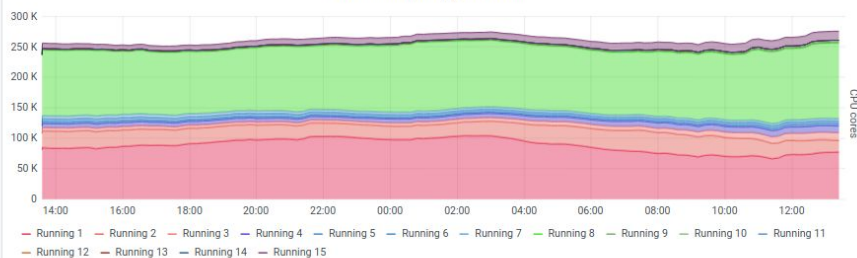
CPUs in use by jobs not in the Global or CERN pools



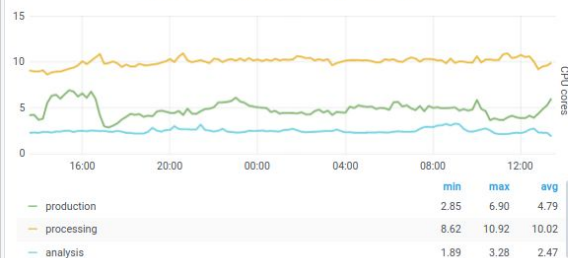
Average cores requested per job



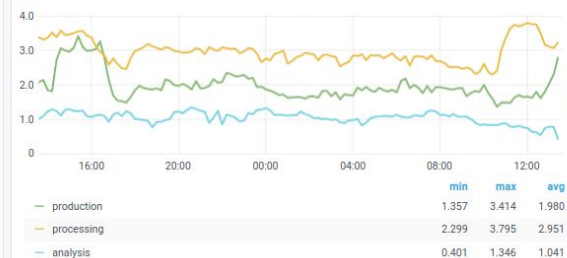
CPU cores for jobs by job core size



Average memory provisioned (GB) per completed job



Average memory used (GB) per completed job



Percentage memory used for completed jobs

